

BACKGROUND

Cheap cameras, and no good way to check

The problem is real

A streaming camera now fits inside a USB charger or a clothes hook and costs less than a meal. Airbnb banned indoor cameras outright in April 2024; South Korea runs municipal inspection teams for public washrooms.

Existing tools don't answer the question

RF detectors (\$20–60) beep at any radio signal — always true indoors, so useless. **Lens finders** need the right angle. **Pro gear** works but costs \$1k–10k and needs training.

They tell you a signal exists. We wanted to know: **is something recording here, right now?**

WHAT WE BUILT

One ESP32, listening passively

Sense

An ESP32 counts bytes per device every 200 ms and splits upload from download. It joins nothing and decrypts nothing — 802.11 headers only.

Judge

~100 lines of Python on a laptop looks for the shape of a camera: heavy one-way upload that reacts when the room changes.

Report

A live dashboard flags suspects, and states UNKNOWN whenever it cannot see properly. It never says “safe.”

DEMO 1

Total radio cost: **one dev board** — no capture card, no SDR.

It sees every device in the room

49

DEVICES FOUND

In a 26-second sweep across all 13 channels

200_{ms}

UPDATE RATE

Exact across 59 windows, zero dropouts

0

PACKETS DECRYPTED

Never joins a network; header fields only

Listing devices is the easy part — phone apps do it. The value is picking out **which one is watching this room.**

The camera gives itself away

Signal 1 — it only uploads

Phones and laptops mostly pull data down. A streaming camera does the opposite, and it never stops.

478 KB/s up · 0 down

170× the next busiest device in the room

Signal 2 — it reacts to the room

Cameras use variable-bitrate encoding: change what the camera sees and you change how much it sends.

Flip the room's light in a known rhythm and see whose traffic follows it. That is how you tell a camera watching *this* room from someone uploading a video next door.

For this demo the board hosts its own 802.11n network, because its radio cannot demodulate WiFi 6. A real camera sits on the room's own network — that limit is on the summary slide.

We scan BLE too — and keep it out of the verdict

What it adds

A second board lists nearby Bluetooth devices — signal, maker, name, and whether the address is randomized. Useful context about what else is in the room.

Why it can't find cameras

Bluetooth carries orders of magnitude less data than live video, so a streaming camera's payload never appears there. Seeing BLE only tells you a device *has* a Bluetooth radio.

Hardest part: phones rotate their Bluetooth address every ~15 minutes, so a naive list fills up with devices that no longer exist. Entries expire fast so the list reflects **what is here now**.

[截图位] BLE 扫描界面

The same signal, other rooms

Confidential meetings

Sweep before a sensitive discussion for anything streaming out of the room.

Exams and classrooms

Sustained one-way upload from a seat looks different from ordinary browsing — without inspecting any content.

Rentals and changing rooms

A routine check between guests, instead of waiting for someone to complain.

All of it reuses one primitive: **who is sending far more than they receive, and does it react to this room?**

Directions, not shipped features.

What works, and what doesn't

Works

Live over real RF, a streaming camera is separated from every other device in the room by **three orders of magnitude** — on upload asymmetry alone.

Blind spots we measured

Cameras recording to an **SD card** or over **cable** emit nothing. **5 GHz and WiFi 6** are outside this radio — we recovered 0.03% of an 802.11ax link. **RSSI is not distance**; we don't localize.

A sweeper that says UNKNOWN when it cannot see is more useful than one that says SAFE when it guessed.

Prior art exists (DeWiCam, SnoopDog, Lumos). Ours is the low-cost build, the entry-sweep scenario, and honest numbers for where it fails.
github.com/DavidWang1231/WashroomSweep